BCM Developer Test — Answers

Q1 — PHP

Bugs - AI Assisted

1. Missing new keyword. $q = WP_Query(...) — WP_Query is a class. Without new, this is a fatal error and the script dies immediately.

2. Wrong pagination key. 'page' => $page is silently ignored. The correct key is 'paged'. All requests return page 1.

3. Unsanitised $_GET['s']. Used directly without an existence check or sanitisation. Absent s causes a PHP notice; present s passes raw user input into the query.

4. Undefined $pages. <?= $pages ?> — $pages is never defined. The page count renders as blank or zero.

5. Missing closing tag / unguarded $_GET['s']. <?= esc_html($_GET['s']) has no closing ?> and assumes s is always set. Results in a parse error and a potential undefined-index notice.

6. Wrong regex pattern. '#[ad]#' is a character class that matches the single letter a or d, not the literal string [ad]. Every a and d in the content gets replaced.

7. echo inside preg_replace_callback. The callback must return the replacement string. Using echo outputs directly (breaking output order) and returns null, so $content becomes empty.

8. esc_html(the_title()). the_title() echoes by default when called without arguments. Wrapping it in esc_html() causes the title to print twice, once raw from the echo, once escaped from esc_html. Recommended to use get_the_title() instead.

9. No wp_reset_postdata(). After a custom WP_Query loop, the global post object is left pointing at the last custom post. Any subsequent template code gets the wrong data.

Assumptions about [ad] markers

The marker is [ad] or [ad:variant] (e.g. [ad:leaderboard]). The replacement div is a client-side placeholder; no content is injected server-side.

Key changes in the corrected file (q1-php/posts.php)

- new WP_Query()
- 'paged' key instead of 'page'
- sanitize_text_field() on $_GET['s'], guarded with isset
- $pages = $q->max_num_pages
- Pattern /\[ad(?::([a-zA-Z0-9_-]+))?\]/ — matches the literal brackets and captures an optional variant
- Callback accepts $m and returns the div string
- get_the_title() instead of the_title()
- wp_kses_post($content) to allow the injected div while stripping unsafe HTML
- wp_reset_postdata() at the end

Checks

Scenario	Expected behaviour
No ?page=	Defaults to 1; always a positive int
?page=2	Returns posts 11–20; $pages shows total page count
?s=xyznotexist	have_posts() is false; "No results found." shown
Multiple [ad] markers	Both replaced; [ad:leaderboard] gets class ad-leaderboard

Q2 — Editor Cannot Upload Images

By default, I will start by reproducing the problem. log in as the editor, upload a small, ordinary JPEG, then have the colleague upload the editor's original file. This separates a problem with the file from a problem with the account, and it decides the order of everything else. Otherwise, I will then proceed to do the following;

1. Editors normally have the upload_files permission, but a role plugin, membership plugin, or custom code can remove it. I will compare the editor's capabilities with the colleague's using a capability viewer such as User Role Editor, or WP-CLI. If upload_files is missing, I will restore it in the role settings and find a fix to whatever stripped it.

2. If the small JPEG works but the editor's file fails, I will look at the exact error text.

- "Sorry, you are not allowed to upload this file type" points to an unsupported format such as HEIC or SVG.
- "Exceeds the maximum upload size" points to size limits. This can be fixed by converting or compressing the image.
- If the format or size is something the site should accept, allow the file type deliberately or raise upload_max_filesize and post_max_size.

3. A firewall, ModSecurity rule, security plugin, or CDN maybe rejecting the user's requests. Opening the browser developer tools Network tab during a failed upload to look at the status code on the request for async-upload.php or the REST media endpoint. A 403 or 406 suggests a block.

4. A media-folder or offload-to-cloud plugin may have per-user settings, and folder permissions or low disk space could be at fault. Disk and permissions problems usually affect everyone, so I check them last.

Q3 — Slow Article Page on Mobile

Before changing anything, I would take a baseline using PageSpeed Insights (including real-user field data) and a throttled mobile run in WebPageTest. Then firstly;

1. Run Image optimization. This means serving correctly sized responsive images in WebP or AVIF, and lazy-loading everything except the main image at the top. First measure Largest Contentful Paint, total page weight, and which element is the LCP.

2. Implement Page caching and a CDN. Serving pre-built HTML from cache avoids running PHP and database queries on every view. I will measure server response time (TTFB) too.

3. Cutting third-party scripts and unused plugin code.** This means deferring non-critical scripts and removing CSS and JS that plugins load on every page. First measure Total Blocking Time and Interaction to Next Paint, and use the request waterfall to see which scripts cost the most.

For Caching approach,

- Ads: It is important to confirm that ad requests are still firing and they render on a cached page. Ad scripts that depend on per-visitor data must not be frozen into the cached HTML.

- Analytics: Confirm the analytic tags fires using Tag Assistant or the Network tab, and check the pageview in real-time reports. Also, check it is not double-counted, and compare daily pageview totals against the days before the change.

- Signed-in behaviour: Log in, then compare an incognito view with a signed-in view. The cache should be bypassed for the logged-in cookie, so the admin bar, comment forms, personalised content etc behave as before, and one user's version is never served to another. Checking response headers (Cache-Control and the cache plugin or CDN's hit/miss header) confirms which requests came from cache.

Q5 — HTML Corrections

Validity errors (spec violations)

Error	Fix
</html> before <body>	Moved </html> to end of document
href=https://...	Add the quote to the URL: href="https://www.bigcabal.com"
 — space before >	Removed the space
<abbr name="lobortis">	Changed to title="lobortis"
<img src=test.jpeg with=300 height=auto>	Quoted the src, and other fixed typos. src="test.jpeg" width="300" alt="..."
<code>...</code> — overlapping tags	Corrected to <code>...</code>
Missing <!DOCTYPE html>	Added
Missing charset meta	Added <meta charset="UTF-8">

Accessibility improvements

- alt on <img>: screen readers otherwise announce the filename
- lang="en" on <html>: correct pronunciation for screen readers and language indexing for search
- type="button" on <button>: without it a button defaults to type="submit" inside a form

HTML-only improvements for search discoverability

1. <meta name="description">: Used by search engines for result snippets

2. <main> and <article>: <article> signals the primary content body. search engines give it higher relevance weight than a generic
3. lang="en" on <html>: correct language identification affects multilingual indexing and regional search relevance

Q6 — JavaScript

Checks

Scenario	Result
Initial load (3 paras)	No HR (fewer than 3 items before index 2)
One click (6 paras)	One HR before index-2 paragraph
Two clicks (9 paras)	HRs at indices 2 and 4
Toggle before adding	Image para hidden; new image clones also hidden
Toggle after adding	All image paras shown
Toggle on, then add	New image clones hidden
Hidden para at index 2	Still triggers HR (counted regardless of display state)

Q7 — Stale Content After Theme Release

Assumptions: the site sits behind a page cache or CDN, the release went through the normal deployment process, and the correction was saved in WordPress before the release.

First three checks will be;

1. Compare what readers actually receive. Request the article URL from two locations or networks, with and without a cache-busting query string. Compare the response headers like Age, Cache-Control, and the CDN or cache hit/miss header, and check whether the correction text and theme asset versions are in the HTML.

2. Open the post's revision history to confirm the correction is published rather than a draft or scheduled revision. Then fetch the page straight from the origin, bypassing the CDN, to see whether it returns the corrected text.

Telling cache from code or content

- If the origin serves the corrected text and the edge serves the old one, it is a cache problem.
- If different servers return different HTML for the same URL, it is a partial deployment.
- If the origin itself returns the old text, it is a content problem, for example a revision that was never published or a different post edited by mistake.
- If everyone gets the corrected text but something renders wrongly, it is a code problem in the release. Old CSS or JavaScript on some devices points to browser or service-worker caching of theme assets.

When to roll back: only do this if the release itself is causing the fault, such as templates that fail to show updated content or errors that affect many readers, and a purge or quick fix cannot resolve it within a time agreed with the editors. Rolling back a healthy release does nothing for a stale cache.

Verify recovery by fetching the article from several regions on desktop and mobile and compare the HTML and headers. Check that other recently corrected articles are also current, and watch error logs and the cache hit ratio. Review again after the next scheduled update.

Update to the editor

"Some readers are seeing the older version of the corrected article because of what looks like a caching issue after the release, and we are confirming this against the origin now. A fresh release would not clear it, so we are purging the cache instead and will confirm once the correction shows for all readers."

Q8 — Reducing Manual Copy-Paste Before Publishing

What to ask or observe first

Sit with one editor through a real publish cycle and record what gets copied (title, URL, image), where it goes, where mistakes happen (wrong link, old title, wrong image), and how long it takes.

Smallest useful improvement

A "Copy Data" button in the post editor that writes title, permalink, featured image URL, excerpt, and tags to the clipboard in one click. This is roughly 50 lines of PHP and JavaScript using the WordPress REST API and navigator.clipboard.writeText().

Simpler alternative (no new code): A browser bookmarklet that reads the open post page and copies the same fields. Zero backend changes, zero plugin installation.

Why this choice:

The button lives inside the existing workflow, eliminates every manual selection step, and produces a consistent format every time. The bookmarklet is a fallback if plugin installation is restricted.

What stays under editorial control

Editors still decide when to publish, what copy to use in each tool, and which image to feature. The tool only retrieves existing data faster.

Testing and success

One session alongside one editor, using the button for one publish. Watch for format mismatches and missing fields. Success: time-per-publish drops from the observed baseline; zero wrong-URL or mismatched-image errors over two weeks; the editor notices when it breaks.

Prior example

I built a simple quick send Bulk mails and WhatsApp (had jitters implemented to avoid WhatsApp bans) tool for a small team that didn't have a premium CRM tool. I connected Brevo for the email and a self

hosted WhatsApp bridge where they could send messages to their managed Leads / contacts at once, with analytics and tracker.

Q9 — Newsletter Signup Reliability

Approach

1. Validate the email locally first. Invalid addresses never reach the API.
2. Derive an idempotency key from sha256(lower(trim(email))).
3. Check local storage for that key. If found, return the stored result immediately. Safe against any number of retries.
4. Call the API with the key as an Idempotency-Key header. The retry loop handles:
    - Timeout: Retry up to MAX_RETRIES times with linear back-off; return pending if all attempts fail so a background job can retry later.
    - 429 Rate limit: Read Retry-After, sleep (capped at 60s), retry.
    - 422 Invalid email: Return immediately; provider rejected the address.
    - 409 Conflict: Subscriber already exists; store and return already_exists.
5. On success, store the key and result before returning.

Logging

Log: hashed idempotency key, event type, attempt number, Retry-After values.

Do not log: raw email (PII), API response bodies, request payloads.

Testing duplicate prevention

- Simulate a timeout: Send the request, kill the connection after the API processes it but before it responds.
- Call subscribe_user with the same email again.
- Expected: the API returns 409 (or honours the idempotency key), the function records already_exists, and the subscriber exists exactly once in the mock database.

Q10 — Security Review: WordPress Endpoint

The claim is wrong

"The button is visible only to editors" is a client-side control. Any user, authenticated or not, can send a raw HTTP POST directly to the endpoint. Client-side visibility is not a security boundary.

Required server-side checks

1. Authentication — is the request from a logged-in user?
2. Nonce — is a valid single-use WordPress nonce present? (CSRF protection)
3. Capability — does this user have edit_published_posts or equivalent?
4. Input validation — is the post ID a positive integer pointing to an existing post? Is the status one of the permitted values (draft, publish, pending)?
5. Scope — can this editor change any post, or only their own?

Check order (pseudocode)

```
function update_post_status(request):
  if not is_user_logged_in():
    return 401
  if not verify_nonce(request.nonce, 'update_post_status'):
    return 403
  if not current_user_can('edit_published_posts'):
    return 403
  post_id = intval(request.post_id)
  if post_id <= 0 or not post_exists(post_id):
    return 400
  allowed = ['draft', 'publish', 'pending']
  if request.status not in allowed:
    return 400
  wp_update_post({'ID': post_id, 'post_status': request.status})
  return 200
```

Two tests that must be rejected

1. Unauthenticated POST: No session cookie. Must return 401/403; must not update anything.
2. CSRF attempt: Valid editor session cookie, missing or wrong nonce. Must return 403. Simulates an attacker embedding a form on an external site that triggers the request through the editor's browser.

How to raise it

In the code review or directly:
"This endpoint relies on the button being hidden, which can be bypassed with a single curl command. It needs nonce verification, a capability check, and input validation before it ships. I have written the check order above for your perusal."